Real Hardware Effects of Unstructured Pruning on Edge Platforms

D. Aguirre Hirt
Arizona State University
Tempe, USA
dpaguir2@asu.edu

Zachary Stewart
Arizona State University
Tempe, USA
zsstewar@asu.edu

***Abstract*—In this paper, we present a pruning and fine-tuning framework for compressing BERT for deployment on resource constrained hardware. Our method identifies a three-step process to achieve automatic pruning on $\text{BERT}_{\text{BASE}}$, and deployment of this compressed model onto the popular edge device Raspberry Pi 5. We evaluate the approach on GLUE tasks (SST-2), demonstrating strong pre and post-fine-tuning accuracies compared to $\text{BERT}_{\text{Tickets}}$. We provide a theoretical hardware analysis, estimating FPGA speedups relative to CPU and GPU, and show a 30% sparsity leads to a 1.43x theoretical latency improvement on a Xilinx Alveo U200. Our edge deployment on a Raspberry Pi 5 shows how resource constrained hardware compares with larger CPUs and GPUs. These results highlight high viability on pruning driven compression for edge deployments.**

***Index Terms*—pruning, latency, speedup, accuracy**

I. Introduction

Transformer-based models have achieved remarkable success across a wide range of natural language processing (NLP) tasks, including sentiment classification [8], question answering [9], text summarization [10], and language modeling [11]. Among them, BERT [7] has demonstrated substantial improvements on the GLUE benchmark [12], a widely adopted suite for evaluating language understanding systems. As deep learning becomes increasingly part of industry [13], there is growing interest in deploying BERT on resource-constrained devices that require low-latency and high-accuracy inference. Prior work has explored identifying sub-networks of the $\text{BERT}_{\text{BASE}}$ model with desirable sparsity and accuracy [14]–[16], but these approaches rely on expert-designed heuristics that are often time-consuming and fail to yield globally optimal solutions [17]. Automatic weight is computationally expensive, requiring repeated sub-network sampling and multi-epoch retraining. Moreover, pruning Transformer models remains particularly challenging due to the sensitivity of syntax and semantics in language tasks.

Motivated by these limitations, we introduce a simple pruning framework for BERT that samples candidate sub-networks, evaluates them without fine-tuning, and subsequently fine-tunes the most promising candidates. Our approach aims to identify high-quality sub-networks without requiring expert intervention and eliminates the need for the thousands of training epochs typical of existing automatic pruning methods [18], [33]. Our contributions are as follows: (1) We implement an efficient pruning framework (BERT-A2) that identifies sparse sub-networks without costly iterative retraining. (2) We demonstrate competitive accuracy on SST-2 across multiple sparsity levels, significantly improving pre-fine-tuning accuracy over prior work. (3) We provide a hardware-oriented analysis, including theoretical CPU→FPGA and GPU→FPGA speedups, establishing upper-bound performance expectations for edge deployment.

II. Related Work

Transformer-based Models. Transformer architectures have achieved state-of-the-art performance across major NLP tasks such as question answering [25], machine translation [26], and speech recognition [27]. Their success is largely driven by the multi-head self-attention mechanism, which models long-range dependencies in parallel [20]. Transformers have also expanded into vision through models like ViT [28] and iGPT [19]. However, large models such as $\text{BERT}_{\text{BASE}}$ (110M parameters) and $\text{BERT}_{\text{LARGE}}$ (340M parameters) demand substantial compute and memory, limiting their deployment on edge devices. Recent compact models such as TinyLlama demonstrate the growing need for highly efficient Transformer variants tailored for low-resource hardware.

BERT Pruning. Weight pruning has emerged as a practical strategy to shrink BERT while preserving accuracy. Early methods include irregular magnitude pruning (IMWP), which removes 30–40% of low-magnitude weights [23], and irregular reweighted proximal pruning (IRPP), which increases pruning ratios by 59.3% without accuracy loss [24]. Other studies examine global and task-specific redundancy in BERT and XL-Net [21], [22]. Inspired by the Lottery Ticket Hypothesis [1], recent work [14]–[16] identifies sparse BERT sub-networks that retain competitive performance. Additional pruning approaches [29]–[31] further reduce model size and improve inference speed, enabling more practical deployment on edge and resource-constrained platforms.

III. Methodology

A. Problem Formulation

We base on our implementation on AE-BERT from [3]. Since there was no public code and the authors could not share the code we did our own implementation (BERT-A2) and improved based [3].

Pruning Strategy. For an N-layer BERT model with parameters $\{W_i, b_i\}_{i=1}^N$, a pruning strategy is

$$B_j = (\beta_1^j, \beta_2^j, \ldots, \beta_N^j), \quad j = 1, \ldots, S$$

where β_i^j is the pruning ratio of layer i, with a sparsity s. We seek to randomly generate S pruning strategies. The magnitude pruning mask will generate the pruned weights:

$$W_i^{(j)} = \text{Prune}(W_i).$$

BERT-A2 Optimization (Eq. 1). The idea behind the algorithm is to find the best pruning strategy that yield a model(BERT) with the best accuracy in a simple evaluation step. Thus Eq.1 becomes an optimization problem where we try to find the optimal pruned sub-network such that the loss function is minimized and the performance of the model is the best.

$$B^* = \arg\min_{B_j} f\left(\{W_i\}_{i=1}^N, \{b_i\}_{i=1}^N, B_j\right), \qquad j = 1, \ldots, S.$$

For solving the optimization problem we propose the following three stages:

1) **Generation:** Produce pruning strategies that satisfy the imposed constraints and apply them to the original full model.
2) **Sub network Evaluation (Stage II).** Find the accuracy of the resulting pruned sub-networks without performing any fine-tuning. The sub-network evaluation is fine-tuning free and evaluated on less than 5 epochs.

$$\text{Eval}(M_j) = \frac{1}{|D|} \sum_{(x,y)\in D} \mathbf{1}\left[\hat{y} = \arg\max \text{BERT}_{M_j}(x)\right].$$

Select the best subnet

$$j^* = \arg\max_j \text{Eval}(M_j).$$

3) **Fine-tuning (Stage III)** Select the top 5 percent of the best candidates with the highest evaluation score and fine-tune it to obtain the final optimized model.

$$M^* = \arg\min_M \mathcal{L}_{\text{task}}(M).$$

IV. Experimental Evaluation

A. Datasets

We adopt the GLUE benchmark [4] as our evaluation suite. GLUE comprises multiple tasks grouped by objective and difficulty, including single-sentence classification, and natural language inference. In our experiments, we evaluate the proposed method on one single-sentence sentiment classification similarity task—SST-2 [6]

B. Experiment Setup

The pruning and fine-tuning experiments are conducted using PyTorch and HuggingFace Transformers using an NVIDIA A100 GPU paired with an Intel Xeon Gold 5128 (2.30 GHz) CPU. To prune, we sample 20 candidate pruning strategies with a fixed sparsity. The top 5% are fine-tuned for two epochs using AdamW with a learning rate between 2×10^{-5} and 5×10^{-5}, and the candidate with the best validation accuracy for each task is selected. Pruning is only applied to encoder weights. The best 5 percent of pruned models is fine-tuned.

C. Baseline Models

Our baseline is a fine-tuned, unpruned $BERT_{\text{BASE}}$ model [7]. We report the performance of the official `bert-base-uncased` checkpoint as Model with sparsity, s=0.

All experiments are conducted using the HuggingFace Transformers library [32]. The model consists of 12 encoder layers ($L = 12$), a hidden size of $H = 768$, and 12 self-attention heads ($A = 12$), totaling approximately 110 million parameters. In addition, we compare our framework against one published compression baselines: $BERT_{\text{Tickets}}$ [1].

D. Metrics

We follow the GLUE evaluation protocol [4]: accuracy score for SST-2. Latency (ms) and throughput (samples/s) are calculated in hardware evaluation for a comparison of CPU, GPU, Raspberry Pi 5, and theoretical FPGA deployment.

E. Platforms

Software experiments are performed using Python 3.11, PyTorch 2.1.0, and CUDA 11.8 on an NVIDIA A100 GPU paired with an Intel Xeon Gold 5128 (2.30 GHz) CPU. For hardware evaluation, we target the Xilinx Alveo U200 FPGA, which includes 4,320 18k BRAMs, 6,840 DSPs, and 1,882.2 logic cells LUTs, and supports C/C++ high-level synthesis. To report theoretical hardware inference performance, we additionally compare against an Intel Xeon Gold 5128 (2.30 GHz) CPU, reporting latency and throughput (frames/sequences per second, samples per second).

F. Hyperparameter Selection

During fine-tuning, we train for two epochs with a batch size of 32 and a learning rate between 2×10^{-5} and 5×10^{-5}, selecting the value that yields the best validation accuracy for each task. For pruning, we randomly generate 20 pruning strategies that satisfy a fixed overall sparsity target. Pruning is applied to encoder weights only. For SST-2, we enforce overall pruning ratios of 0.3 and 0.7, respectively. Finally, the best 5 percent of pruned models is selected and fine-tuned.

G. Implementation Details

We implement our version of AE-BERT (BERT-A2) using Python, PyTorch, and the HuggingFace Transformers library. All scripts for pruning, fine-tuning, hardware evaluation, and latency benchmarking are available at: https://github.com/bosho86/BERTA2.git.

H. Results

To benchmark the accuracy of our compression algorithm implementation we ran [1] with the $\text{BERT}_{\text{BASE}}$. Table I shows that the pre-fine-tuning accuracies are recovered after a fine-tuning step for $\text{BERT}_{\text{Tickets}}$ [8] and Table II shows our implementation BERT_{A2}. $\text{BERT}_{\text{BASE}}$ conducts their pruning prior to initial training, then fine tune following the prune. Our model BERT_{A2} prunes following training, then selects the best candidates. Thus, our accuracies pre-fine-tuning are much better than [1] and after the fine-tune steps they are comparable. The pruning strategy optimization problem assures we find the best candidate in terms of accuracy even before the pre-tuning step.

TABLE I
SST-2 ACCURACY AND EVALUATION LOSS OF DENSE AND PRUNED MODELS FOR [1]

Implementation	Sparsity	Pre-FT Acc	Post-FT Acc	FT Loss
$\text{BERT}_{\text{BASE}}$	0	0.4920	0.926	0.278
$\text{BERT}_{\text{Tickets}}$	0.3	0.509	0.930	0.6586
$\text{BERT}_{\text{Tickets}}$	0.7	0.509	0.930	0.289

Next, we used the SST-2 instruction on the GLUE benchmark with BERT-A2. From Table II we found that at a higher sparsity the pre-fine-tuning accuracy was significantly recovered by the fine-tuning step. On the other hand at a lower sparsity, the pre and post-fine-tuning accuracy remained rather similar.

TABLE II
SST-2 ACCURACY AND EVALUATION LOSS OF BERT-A2 DENSE AND PRUNED MODELS

Sparsity	Fine-Tuning Performance			
	Pre-FT Acc	*Post-FT Acc*	*FT Loss*	*Avg Prune Ratio*
0.3	0.992	0.930	0.251	0.300
0.7	0.820	0.880	0.294	0.646

Pre-FT = pre-fine-tuning, Post-FT = post-fine-tuning.

We believe that the accuracy degradation at higher sparsitites is due to the Layer-wise sparsity distribution, where the attention Q/K/V seems to have an effective higher sparsity ratio than in the other parts of the encoder (see Fig. 1). In general terms, the accuracy degradation in BERT is strongly correlated with the amount of pruning applied to the Q/K/V projection matrices in the first three encoder layers. These early layers are responsible for establishing the fundamental token–to–token dependencies used throughout the network; therefore, aggressive sparsification disrupts the formation of reliable attention patterns [7]. When sparsity exceeds 30–40% in these layers, we observe a sharp decline in accuracy indicating that later layers cannot compensate for corrupted early contextual representations [5]. In contrast, pruning deeper layers or the feed-forward blocks results in significantly smaller accuracy drops. These findings confirm that early-layer attention projections are structurally critical, and high sparsity in their QKV components can cause accuracy losses.

Table. III shows the latency and throughput of our implementation BERT_{A2}.

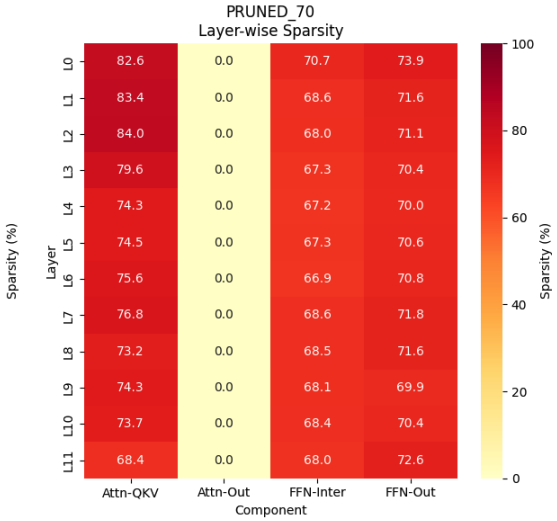

Fig. 1. Layer sparsity distribution for BERT-A2

TABLE III
COMPARISON OF INFERENCE PERFORMANCE ON DIFFERENT HARDWARE DEVICES FOR BERT-TICKET [1] AND OUR IMPLEMENTATION (BERT-A2).

Sparsity	Implementation	Hardware Performance		
		Latency (ms/sample)	Throughput (samples/s)	Device
$s = 0$	$\text{BERT}_{\text{BASE}}$	232.64	4.30	CPU
$s = 0.3$	$\text{BERT}_{\text{Tickets}}$	230.51	4.34	CPU
$s = 0.3$	BERT_{A2}	218.23	4.58	CPU
$s = 0.7$	$\text{BERT}_{\text{Tickets}}$	232.04	4.31	CPU
$s = 0.7$	BERT_{A2}	218.18	4.58	CPU
$s = 0$	$\text{BERT}_{\text{BASE}}$	1.46	682.81	NVIDIA A100
$s = 0.3$	$\text{BERT}_{\text{Tickets}}$	1.46	686.29	NVIDIA A100
$s = 0.3$	BERT_{A2}	1.60	626.07	NVIDIA A100
$s = 0.7$	$\text{BERT}_{\text{Tickets}}$	1.46	686.17	NVIDIA A100
$s = 0.7$	BERT_{A2}	1.60	625.85	NVIDIA A100

Our implementation reduces latency time and increases throughput, when compared to [1] ($\text{BERT}_{\text{Tickets}}$) and $\text{BERT}_{\text{BASE}}$. The pruning strategy optimization and pruning mask, works as a dynamical pruning yielding better compression algorithms. Finally, for the NVIDIA A100 our implementation has comparable results with $\text{BERT}_{\text{Tickets}}$

1) Knowledge Distillation: Knowledge distillation transfers information from a large teacher model to a smaller student model by training the student to match the teacher's softened probability distribution. This allows the student to approach the teacher's accuracy despite having fewer parameters. Prior works applied knowledge distillation (KD) to transfer information from the full BERT model to shallow sub-networks with fewer encoder layers. In contrast, [34] and [35] employed KD to transfer knowledge from the dense BERT model to sparse sub-networks that retain the same number of encoder layers. Moreover, [35] demonstrated that layer-wise knowledge distillation effectively reduces overfitting and improves accuracy during fine-tuning of compressed models. We will like to add further step of KD into our previous pruning algorithm to improve our post-fine-tuning accuracy.

2) Hardware Evaluation and Analysis: Currently, we have performed a theoretical benchmarking using our CPU results

to predict the speedup if we deployed the compressed BERT model onto one of SOL's FPGAs (Xilinx Alveo U200 FPGA board). We use the 9.97 ms dense BERT latency reported by Rizk et al. [34] as our proxy baseline, since their evaluation targets a BERT model with the same architectural configuration (layers, hidden size, and attention heads) and is implemented on a closely related Virtex UltraScale+ FPGA family device. Using this baseline, we estimate the pruned FPGA latency as:

$$\text{FPGA}_{\text{latency}} = \text{FPGA}_{\text{baseline}} \times (1 - s),$$

where s is the sparsity. The theoretical speedup is:

$$\text{Speedup} = \frac{\text{FPGA}_{\text{baseline}}}{\text{FPGA}_{\text{latency}}}.$$

Using the above model, we compute the expected latency improvement when applying 30% unstructured sparsity to $\text{BERT}_{\text{Tickets}}$ and our BERT_{A2} model. Table IV summarizes the theoretical latency and speedup assuming ideal sparsity exploitation and no bandwidth or memory-access penalties.

TABLE IV
THEORETICAL FPGA LATENCY AND SPEEDUP FOR DENSE, 30%, AND 70% SPARSE BERT MODELS ON A XILINX ALVEO U200, ASSUMING IDEAL LINEAR SCALING WITH SPARSITY BASED ON RIZK ET AL. [34].

Sparsity	Implementation	FPGA Latency (ms/sample)	Speedup vs. Dense (×)
$s = 0$	$\text{BERT}_{\text{BASE}}$	9.97	1.00
$s = 0.3$	$\text{BERT}_{\text{Tickets}}$	6.98	1.43
$s = 0.3$	BERT_{A2}	6.98	1.43
$s = 0.7$	$\text{BERT}_{\text{Tickets}}$	2.99	3.33
$s = 0.7$	BERT_{A2}	2.99	3.33

These results suggest a BERT model deployed on a Xilinx Alveo U200 FPGA yields a 1.43× acceleration with a 30% sparsity. The estimated 6.98ms shows a meaningful reduction; the actual speedup would depend on multiple factors: synthesis flow, memory bandwidth, mapping of the compressed model to FPGA resources, amongst many others. Furthermore, our theoretical deployment yields a 3.33x acceleration with a 70% sparsity. Thus, our theoretical deployment should be interpreted as a conservative upper bound on FPGA performance and serves primarily as a comparative reference against our CPU and GPU measurements.

To relate the theoretical latencies and speedup for dense, 30%, and 70% sparse BERT models on a Xilinx Alveo U200 to our measured CPU and GPU baselines, we compute the relative latency speedups achieved when moving each device to a projected FPGA deployment. Table V demonstrates how sparsity driven acceleration on FPGA scales with respect to Intel Xeon Gold 5128 (2.30 GHz) CPU and NVIDIA A100 GPU.

In addition to latency, throughput also provides an insightful measure in assessing performance efficiency. Using the reciprocal of the projected FPGA latency, we derive theoretical FPGA throughput and compare it against measured CPU and GPU throughput. This shows how sparsity affects the sustained processing rate across our CPU and GPU devices.

TABLE V
THEORETICAL FPGA LATENCY AND RELATIVE LATENCY SPEEDUPS VS. CPU AND NVIDIA A100 GPU.

Sparsity	Impl.	FPGA Lat. (ms)	CPU→FPGA	GPU→FPGA
$s = 0$	$\text{BERT}_{\text{BASE}}$	9.97	23.33×	0.15×
$s = 0.3$	$\text{BERT}_{\text{Tickets}}$	6.98	33.02×	0.21×
$s = 0.3$	BERT_{A2}	6.98	31.27×	0.23×
$s = 0.7$	$\text{BERT}_{\text{Tickets}}$	2.99	77.6×	0.49×
$s = 0.7$	BERT_{A2}	2.99	73.0×	0.54×

TABLE VI
THEORETICAL FPGA THROUGHPUT AND RELATIVE THROUGHPUT SPEEDUPS VS. CPU AND NVIDIA A100 GPU.

Sparsity	Impl.	FPGA Thr. (samples/s)	CPU→FPGA	GPU→FPGA
$s = 0$	$\text{BERT}_{\text{BASE}}$	100.30	23.33×	0.15×
$s = 0.3$	$\text{BERT}_{\text{Tickets}}$	143.27	33.01×	0.21×
$s = 0.3$	BERT_{A2}	143.27	31.28×	0.23×
$s = 0.7$	$\text{BERT}_{\text{Tickets}}$	334.45	77.6×	0.49×
$s = 0.7$	BERT_{A2}	334.45	73.0×	0.54×

Overall, the throughput projections show the same inference as for latency: structured sparsity provides a proportional increase in theoretical FPGA performance under an ideal compute-bound assumption.

V. RASPBERRY PI 5 DEPLOYMENT AND ON-DEVICE EVALUATION

1) Deployment Environment: Edge deployment was executed on a Raspberry Pi 5 single board computer which features a Broadcom BCM2712 quad-core Arm Cortex A76 SoC at 2.4GHz, using LPDDR4X-4267 SDRAM, booting from a SanDisk Ultra 32GB microSDHC UHS-I card. The device ran on 64-bit Raspberry Pi Operating System, with python 3.13.5.

We installed a python environment (bertenv) containing onnxruntime, numpy, and psutil. All BERT interface was performed on the Broadcom BCM2712 SoC.

2) Model Conversion and Optimization: The exported BERT dense and pruned models were loaded in ONNX format and fixed with a batch size of 1 with a token sequence length of 11, following the BERT WordPiece tokenizer. All weights and attention blocks were preserved during export to ensure full correctness before applying any compression techniques.

3) On-Device Inference Latency: We evaluated end-to-end inference latency for the dense and pruned BERT models directly on the Raspberry Pi 5 device using ONNX runtime and CPU execution provider. Latency was measured as the wall-clock time required to tokenize the input, run a forward pass through the ONNX graph, and compute the final predicted class. Table VII shows the measured inference latency for Dense-0, Pruned-30%, and Pruned-70% models. Pruning does increase parameter sparsity, however, unstructured pruning does not reduce tensor dimensions in linear layers, resulting in a low variance in compute workload for the models.

4) Throughput and Constraints: Table VIII shows resulting throughput values for the Dense-0%, Pruned-30%, and Pruned-70% models. Consistent with the latency results, all three models achieve similar throughput. This is expected because unstructured pruning does not reduce tensor dimensions and

TABLE VII
RASPBERRY PI 5 ON-DEVICE INFERENCE LATENCY

Model	Latency (ms)
Dense-0% Sparsity	256.24 ms
Pruned-30% Sparsity	274.26 ms
Pruned-70% Sparsity	259.11 ms

ONNX runtimes CPU backend performs dense matrix multiplications regardless of weight sparsity.

TABLE VIII
RASPBERRY PI 5 ON-DEVICE THROUGHPUT

Model	Throughput (samples/s)
Dense-0% Sparsity	3.90 samples/s
Pruned-30% Sparsity	3.65 samples/s
Pruned-70% Sparsity	3.86 samples/s

5) Summary and Deployment Feasibility: Many model compression techniques reduce memory footprint, but do not inherently improve runtime latency. Latency benefits only appear when the underlying hardware or runtime is sparsity-aware. As a result of deploying on Raspberry Pi 5 edge device, we directly show this limitation. As unstructured pruning does not reduce tensor dimensions in linear layers, the ONNX runtime executes the pruned models using the same dense matrix operations as the baseline. Thus, our deployment proves that unstructured pruning alone does not produce speedups on edge devices without sparse kernels.

VI. RASPBERRY PI 5 AND NVIDIA A100 COMPARISON

Figure 2 shows slight pruning from dense to 30% yields a potential slight improvement in accuracy, and thus can be concluded light, unstructured magnitude based pruning may lead to comparable accuracies with small variance. The heavier the pruning (more than 30%), is where accuracy heavily drops. This is due to possibly essential weights in the network being removed, thus leading to inaccurate predictions.

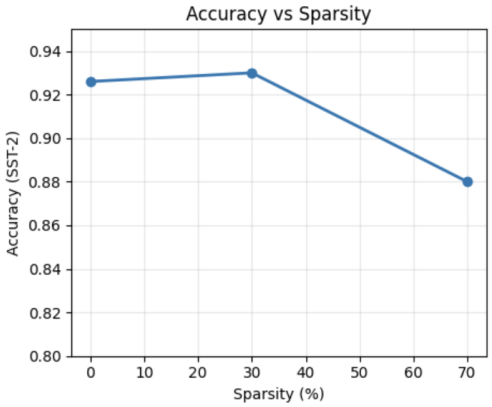

Fig. 2. Raspberry Pi 5, A100 Accuracy vs Sparsity

Figure 3 shows latency versus sparsity, shows end-to-end inference latency on an NVIDIA A100 GPU and Raspberry Pi 5. GPU dominates inference latency calculations as transformer layers like BERT are dominated by matrix multiplication and batched attention, allowing the GPU to leverage its many cores to compute multiply-accumulate operations in parallel. Our Raspberry Pi 5 device uses a CPU, with significantly less memory bandwidth and fewer multiply-accumulate units, which can not be deeply parallelized. Thus, CPU devices sit around 200-300ms/sample, while GPU devices sit around 1-3ms/sample.

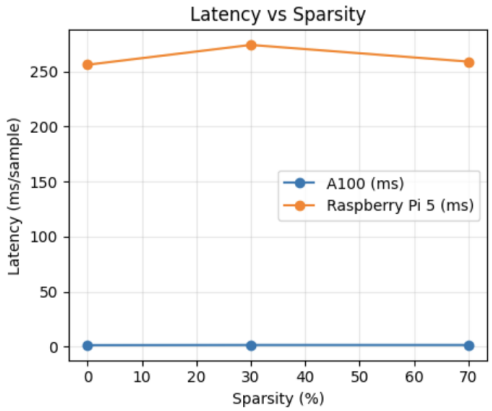

Fig. 3. Raspberry Pi 5, A100 Latency vs Sparsity

Figure 4 visualizes speedup versus the dense baseline, our baseline model $\text{BERT}_{\text{BASE}}$, speedup is observed on the NVIDIA A100 and Raspberry Pi 5. Speedup sharply decreases for both devices as sparsity increases to 30%, following 30% sparsity, the speedup on the Raspberry Pi 5 trends upward, while the A100 speedup stays constant and does not significantly increase or decrease as sparsity increases. This is as expected, A100 GPU does not benefit from unstructured pruning patterns due to masks not being compressed in matrix multiplication, thus the kernel still computes even though the computation is zero multiplication. The Raspberry Pi 5 trend is expected as well, initial magnitude pruning drops speedup as cache is filled now with useless zeros, and doesn't recover until sparsity increases. This is due to tensors getting filled with more zeros, thus the CPU leverages cache behavior and memory access to maximize efficiency.

Table IX highlights the summary of accuracy, latency, throughput, and speedup for dense and pruned models across devices. This summary highlights tight similarities in regards to device (A100, Raspberry Pi 5), which demonstrates how unstructured magnitude pruning behaves using the GLUE benchmark on the SST-2 dataset. Using a strong GPU as NVIDIA A100 allows massive parallelization leveraging matrix multiplication and multiply-accumulation operations, which prompt a low latency during the experiment, and yields a large throughput. Furthermore, on CPU devices and our edge deployment Raspberry Pi 5, Latency is significantly larger than NVIDIA A100 GPU, as well as throughput is substantially smaller. Finally, we recognize unstructured pruning patterns due to masks not being compressed in matrix multiplication, thus the kernel still computes even though the computation is

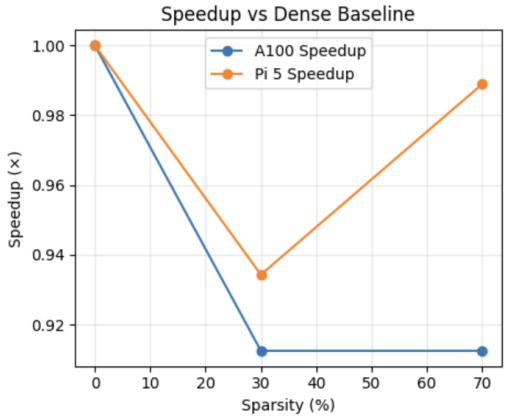

Fig. 4. Raspberry Pi 5, A100 Speedup vs Dense Baseline

TABLE IX
HARDWARE EVALUATION SUMMARY: ACCURACY, LATENCY, THROUGHPUT, AND SPEEDUP FOR DENSE AND PRUNED MODELS ACROSS DEVICES.

Sparsity	Accuracy (SST-2)	A100 Lat. (ms)	A100 Thr. (samples/s)	Pi 5 Lat. (ms)	Pi 5 Thr. (samples/s)	A100 Speedup (×)
0%	0.926	1.46	682.81	256.24	3.90	1.00
30%	0.930	1.60	626.07	274.26	3.65	0.91
70%	0.880	1.60	625.85	259.11	3.86	0.91

zero multiplication, hence the A100 speedup stays constant and does not significantly increase or decrease as sparsity increases.

VII. DISCUSSION AND LIMITATIONS

Table I reports the pre–fine-tuning accuracies for the baseline method [1], and Table II presents the corresponding results for our implementation (BERT-A2). The difference in pre–fine-tuning accuracy for the dense model (s=0) arises because, in the baseline [1], the algorithm first prunes the encoder, trains the pruned model, and only then applies fine-tuning. As a result, the encoder is trained using only a fraction of its original weights, which yields a much lower pre–fine-tuning accuracy compared to BERT-A2. In contrast, BERT-A2 first trains the full model and applies pruning only afterward, prior to the fine-tuning stage, resulting in higher accuracies pre fine-tuning.

The latency and throughput of our BERT-A2 could be improved if we could carefully develop an operation scheduling method for hardware design in order to allocate an appropriate amount of resources for each specialized function [35] in a FPGA. If we could have access an FPGA, we would like to map each encoder operation onto the FPGA with parallelism increased only when resources permit.

Then digital signal processor slices (DSP) are hardware units designated for MAC computations like Q/K/V projections and attention dot produces. Next, the Look-Up Table (LUT) is used to build control logic that links the logic between the computation modules. Finally, Flip-flop (FF) could provide state-holding and pipelining for timing and data flow control.

As explained in [35] the process begins with a minimal, non-parallel baseline implementation. The hardware optimization scheme will consist of a scheduler that iteratively identifies the slowest operator—such as sparse matrix multiplications or attention—and allocates additional parallel compute units as long as the DSP, LUT, and FF budgets of FPGA are not exceeded. This incremental approach ensures near-optimal latency while respecting strict resource constraints.

Due to the increasing popularity, cost and ease of use of Raspberry Pi 5 several works [36] - [37] focus on compressing models through pruning and quantization, converting them into lightweight formats that fit the device's limited compute and memory budget. They demonstrate that carefully optimized deep learning architectures can achieve real-time inference on low-power edge hardware, even under realistic traffic and workload conditions.

Therefore, based on the ideas presented in these works we could potentially improve the latency of our pruned BERT by doing profiling of one of the encoders to find the slow ops, give them more threads + NEON-optimized kernels and attempting to keep power/thermal/memory in check. We deploy the pruned BERT/INT 8 quantized model in ONNX Runtime. However, we could use ARM-optimized execution providers (EPs) [36]. By using EPs, ONNX Runtime is configured with multi-threaded execution so that the dominant matrix-multiplication and attention kernels are parallelized across all four Cortex-A76 cores. Because ONNX's GEMM operators are NEON-accelerated, the inference engine automatically exploits the SIMD units, providing the CPU equivalent of increasing DSP parallelism on an FPGA [37]. We additionally apply ONNX graph optimizations—such as operator fusion and constant folding—to reduce memory traffic and minimize latency. This combination of quantization, thread-level parallelism, and NEON-optimized kernels yields substantially lower latency and higher throughput for the pruned BERT encoder on Raspberry Pi 5, in a similar way to the hardware scheduling and resource allocation used in the FPGA implementation.

VIII. CONCLUSION

In this work, we propose a three-step method to achieve automatic pruning on $\text{BERT}_{\text{BASE}}$ and we deploy this compressed model into the popular edge device Raspberry Pi 5. The first part of the compression method randomly applies different pruning strategies and constrains to a base model and sample the sub-networks according to the strategy generated under pruning constraints. The second part evaluates the sampled sub-networks and select the candidate with the best performance on the GLUE, SST-2 dataset. Next, the sub-network with the highest accuracy is selected for the fine tuning stage. Then the winning candidate from the previous stage is selected for fine tuning and it becomes the final sparse model. Experiments on the GLUE SST-2 benchmark tasks show that our proposed method outperforms hand-crafted pruning methods like [1]. Experiments on edge devices show that there is a similar latency inference time of the whole model compared to CPU and the throughput.

IX. Acknowledgment

We would like to thank Dr. Jeff Zhang for his guidance in going in the right direction in this area of research, as well as providing feedback and teaching us the fundamentals of this topic in order to conduct confident research.

References

[1] T. Chen, J. Frankle, S. Chang, S. Liu, Y. Zhang, Z. Wang, and M. Carbin, The Lottery Ticket Hypothesis for Pre-trained BERT Networks, NIPS '20: Proceedings of the 34th International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 2020, pp 15834 - 15846.

[2] Li et al., 'FTRANS: Energy-Efficient Acceleration of Transformers using FPGA', ISLPED 2020.

[3] S. Huang et al., "An Automatic and Efficient BERT Pruning for Edge AI Systems," 2022 23rd International Symposium on Quality Electronic Design (ISQED), Santa Clara, CA, USA, 2022, pp. 1-6.

[4] A. Wang et al., "Glue: A multi-task benchmark and analysis platform for natural language understanding," arXiv preprint arXiv:1804.07461, 2018.

[5] L. Dong, S. Xu, and B. Xu, "Speech-transformer: a no-recurrence sequence-to-sequence model for speech recognition," in International Conference on Acoustics, Speech and Signal Processing. Calgary, Alberta, Canada: IEEE, 2018, pp. 5884–5888.

[6] Richard Socher et al., "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank." in Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, Seattle, Washington, USA, 2013, pp 1631–1642.

[7] J. Devlin et al., "Bert: Pre-training of deep bidirectional transformers for language understanding," in NAACL-HLT (1), 2019

[8] Q.Xieetal.,"Unsuperviseddataaugmentationforconsistencytraining," Advances in Neural Information Processing Systems, vol. 33, 2020.

[9] I. Beltagy et al., "Longformer: The long-document transformer," arXiv preprint arXiv:2004.05150, 2020.

[10] X. Zhang et al., "Hibert: Document level pre-training of hierarchical bidirectional transformers for document summarization," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 5059–5069.

[11] N. Kitaev et al., "Reformer: The efficient transformer," arXiv preprint arXiv:2001.04451, 2020.

[12] A. Wang et al., "Glue: A multi-task benchmark and analysis platform for natural language understanding," arXiv preprint arXiv:1804.07461, 2018.

[13] C. Garvey, "A framework for evaluating barriers to the democratization of artificial intelligence," in Thirty-Second AAAI Conference on Artificial Intelligence, 2018.

[14] T.Chen,J.Frankle,S.Chang,S.Liu,Y.Zhang,Z.Wang,andM.Carbin, "The lottery ticket hypothesis for pre-trained bert networks," in Advances in Neural Information Processing Systems, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 15 834–15 846.

[15] H. You, C. Li, P. Xu, Y. Fu, Y. Wang, X. Chen, R. G. Baraniuk, Z. Wang, and Y. Lin, "Drawing early-bird tickets: Toward more efficient training of deep networks," in International Conference on Learning Representations, 2019.

[16] S. Prasanna, A. Rogers, and A. Rumshisky, "When bert plays the lottery, all tickets are winning," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, 2020, pp. 3208–3229.

[17] D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" arXiv preprint arXiv:2003.03033, 2020.

[18] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in Proceedings of the European Conference on Computer Vision (ECCV), 2018, pp. 784–800.

[19] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever, "Generative pretraining from pixels," in International Conference on Machine Learning. Vienna, Austria: PMLR, 2020, pp. 1691–1703.

[20] A. Vaswani et al., "Attention is all you need," in Advances in neural information processing systems, 2017, pp. 5998–6008.

[21] F. Dalvi, H. Sajjad, N. Durrani, and Y. Belinkov, "Analyzing redundancy in pretrained transformer models," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4908–4926.

[22] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le, "Xlnet: Generalized autoregressive pretraining for language understanding," in Advances in Neural Information Processing Systems, H. Wallach, H. Larochelle, A. Beygelzimer,F. d'Alché-Buc,E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019.

[23] M. A. Gordon, K. Duh, and N. Andrews, “Compressing bert: Studying the effects of weight pruning on transfer learning,” arXiv preprint arXiv:2002.08307, 2020.
[24] F.-M. Guo, S. Liu, F. S. Mungall, X. Lin, and Y. Wang, “Reweighted proximal pruning for large-scale language representation,” arXiv preprint arXiv:1909.12486, 2019.
[25] B. van Aken, B. Winter, A. Löser, and F. A. Gers, “How does bert answer questions? a layer-wise analysis of transformer representations,” in Proceedings of the 28th ACM International Conference on Information and Knowledge Management. New York, NY, USA: ACM, 2019, pp. 1823–1832.
[26] T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” in Proceedings of the ConferenceonEmpiricalMethodsinNaturalLanguageProcessing. Lisbon, Portugal: Association for Computational Linguistics, 2015, pp. 1412– 1421.
[27] L. Dong, S. Xu, and B. Xu, “Speech-transformer: a no-recurrence sequence-to-sequence model for speech recognition,” in International Conference on Acoustics, Speech and Signal Processing. Calgary, Alberta, Canada: IEEE, 2018, pp. 5884–5888.
[28] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2020.
[29] H. Peng, S. Huang, T. Geng, A. Li, W. Jiang, H. Liu, S. Wang, and C. Ding, “Accelerating transformer-based deep learning models on fpgas using column balanced block pruning,” in 2021 22nd International Symposium on Quality Electronic Design (ISQED). IEEE, 2021, pp. 142–148.
[30] S. Huang, S. Chen, H. Peng, D. Manu, Z. Kong, G. Yuan, L. Yang, S. Wang, H. Liu, and C. Ding, “Hmc-tran: A tensor-core inspired hierarchical model compression for transformer-based dnns on gpu,” in Proceedings of the 2021 on Great Lakes Symposium on VLSI, 2021, pp. 169–174.
[31] S. Chen, S. Huang, S. Pandey, B. Li, G. R. Gao, L. Zheng, C. Ding, and H. Liu, “Et: re-thinking self-attention for transformer models on gpus,” in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2021, pp. 1–18.
[32] T. Wolf et al., “Huggingface’s transformers: State-of-the-art natural language processing,” ArXiv, abs/1910.03771, 2019
[33] N. Liu, X. Ma, Z. Xu, Y. Wang, J. Tang, and J. Ye, “Autocompress: An automatic dnn structured pruning framework for ultra-high compression rates,” in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, no. 04, 2020, pp. 4876–4883.
[34] R. Rizk, D. Rizk, F. Rizk, A. Kumar and M. Bayoumi, ”A Resource-Saving Energy-Efficient Reconfigurable Hardware Accelerator for BERT-based Deep Neural Network Language Models using FFT Multiplication,” 2022 IEEE International Symposium on Circuits and Systems (ISCAS), Austin, TX, USA, 2022, pp. 1675-1679, doi: 10.1109/ISCAS48785.2022.9937531.
[35] Shuo Wang and Zhe Li and Caiwen Ding and Bo Yuan and Yanzhi Wang and Qinru Qiu and Yun Liang, ”C-LSTM: Enabling Efficient LSTM using Structured Compression Techniques on FPGAs”, 2018, arXiv
[36] M. B. Musthafa, S. Huda, T. T. Nguyen, Y. Kodera, and Y. Nogami, “Optimized ensemble deep learning for real-time intrusion detection on resource-constrained Raspberry Pi devices,” IEEE Access, 2025.
[37] Salem Ameen and Kangaranmulle Siriwardana and Theo Theodoridis, ”Optimizing Deep Learning Models For Raspberry Pi”, arXiv, 2023.